WFLOW-08: JavaScript & HTTP Actions

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 8 of 10 | **Created:** January 2026 | **Last Updated:** 08/12/2026

Custom Code and API Integration

When built-in actions aren't enough, use JavaScript and HTTP requests for custom integrations. This notebook covers the JavaScript SDK, HTTP request patterns, and common integration scenarios.

Table of Contents

1. JavaScript Action Basics
2. Dynatrace SDK Usage
3. HTTP Request Patterns
4. Error Handling
5. Workflow-Level Failure Branching
6. Working with Data
7. Passing Data Between Tasks
8. Integration Examples
9. Performance Tips
10. Configuring Task Timeouts
11. CMDB-Driven Host Tag Enrichment

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:write` (§11 also needs an API token with `oneAgents.write` stored in the Credential Vault)
Prior Knowledge	**WFLOW-01** through **WFLOW-07**, JavaScript basics

1. JavaScript Action Basics

Action Structure

```
// Every JavaScript action must export a default async function
export default async function(context) {
  // context contains:
  // - event(): trigger event data
  // - result(taskName): previous task results
  // - env: environment secrets

  // Your logic here
  const result = await doSomething();

  // Return value becomes task result
  return {
    output: result,
    success: true
  };
}
```

Accessing Context

```
export default async function({ event, result, env }) {
  // Trigger event data
  const problemTitle = event().title;
  const severity = event().severity;

  // Previous task result
  const previousOutput = result('previous_task').data;

  // Environment secrets
  const apiKey = env.API_KEY;

  return { title: problemTitle };
}
```

Task Configuration

```
name: custom_javascript
type: dynatrace.automations:run-javascript
input:
  script: |
    export default async function({ event }) {
      return { processed: true };
    }
```

2. Dynatrace SDK Usage

Available SDK Clients

Client	Purpose	Import
`queryExecutionClient`	Execute DQL queries	`@dynatrace-sdk/client-query`
`entitiesClient`	Entity CRUD operations	`@dynatrace-sdk/client-classic-environment-v2`
`problemsClient`	Problem management	`@dynatrace-sdk/client-classic-environment-v2`
`eventsClient`	Event ingestion	`@dynatrace-sdk/client-classic-environment-v2`
`settingsClient`	Settings management	`@dynatrace-sdk/client-classic-environment-v2`

Execute DQL Query

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch logs, from: now() - 1h
        | filter loglevel == "ERROR"
        | summarize error_count = count()
      `,
      requestTimeoutMilliseconds: 30000
    }
  });

  const records = result.result.records || [];
  const errorCount = records[0]?.error_count || 0;

  return { error_count: errorCount };
}
```

Get Entity Details

```
import { entitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function({ event }) {
  const entityId = event().root_cause_entity_id;

  const entity = await entitiesClient.getEntity({
    entityId: entityId,
    fields: '+properties,+tags'
  });

  return {
    name: entity.displayName,
    type: entity.type,
    tags: entity.tags || []
  };
}
```

Add Problem Comment

```
import { problemsClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function({ event, result }) {
  const problemId = event().display_id;
  const actionTaken = result('remediation').action;

  await problemsClient.createComment({
    problemId: problemId,
    body: {
      message: `[Automated] Remediation executed: ${actionTaken}`,
      context: 'Workflow automation'
    }
  });

  return { comment_added: true };
}
```

3. HTTP Request Patterns

HTTP Request Action

```
name: call_external_api
type: dynatrace.http:request
input:
  url: "https://api.example.com/endpoint"
  method: POST
  headers:
    Authorization: "Bearer {{ env.API_TOKEN }}"
    Content-Type: "application/json"
  body: |
    {
      "problem_id": "{{ event()['display_id'] }}",
      "severity": "{{ event()['severity'] }}",
      "title": "{{ event()['title'] }}"
    }
```

Using fetch() in JavaScript

```
export default async function({ event, env }) {
  const response = await fetch('https://api.example.com/webhook', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${env.API_TOKEN}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      problem_id: event().display_id,
      severity: event().severity,
      title: event().title,
      timestamp: new Date().toISOString()
    })
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${await response.text()}`);
  }

  const data = await response.json();
  return { response: data };
}
```

GET Request with Query Parameters

```
export default async function({ event, env }) {
  const params = new URLSearchParams({
    entity_id: event().root_cause_entity_id,
    from: new Date(Date.now() - 3600000).toISOString(),
    to: new Date().toISOString()
  });

  const response = await fetch(
    `https://api.example.com/metrics?${params}`,
    {
      headers: {
        'Authorization': `Bearer ${env.API_TOKEN}`
      }
    }
  );

  return await response.json();
}
```

4. Error Handling

Try-Catch Pattern

```
export default async function({ event, env }) {
  try {
    const response = await fetch('https://api.example.com/action', {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${env.TOKEN}` },
      body: JSON.stringify({ id: event().display_id })
    });

    if (!response.ok) {
      const errorText = await response.text();
      return {
        success: false,
        error: `HTTP ${response.status}`,
        details: errorText
      };
    }

    return {
      success: true,
      data: await response.json()
    };

  } catch (error) {
    return {
      success: false,
      error: error.message,
      stack: error.stack
    };
  }
}
```

Retry Logic

```
async function fetchWithRetry(url, options, maxRetries = 3) {
  let lastError;

  for (let attempt = 1; attempt &lt;= maxRetries; attempt++) {
    try {
      const response = await fetch(url, options);

      if (response.ok) {
        return response;
      }

      // Retry on 5xx errors
      if (response.status &gt;= 500 &amp;&amp; attempt &lt; maxRetries) {
        await new Promise(r =&gt; setTimeout(r, 1000 * attempt));
        continue;
      }

      throw new Error(`HTTP ${response.status}`);

    } catch (error) {
      lastError = error;
      if (attempt &lt; maxRetries) {
        await new Promise(r =&gt; setTimeout(r, 1000 * attempt));
      }
    }
  }

  throw lastError;
}

export default async function({ event, env }) {
  const response = await fetchWithRetry(
    'https://api.example.com/action',
    {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${env.TOKEN}` }
    }
  );

  return await response.json();
}
```

5. Workflow-Level Failure Branching

`try` / `catch` inside a single JavaScript task handles **recoverable** errors — a transient API failure you can retry, a missing field you can default. **Workflow-level failure branching** handles the case where an entire task has failed and you want to route around it: run a cleanup, send a failure notification, or skip downstream work.

The mechanism is the task **`conditions`** block. Each task can declare which predecessor states must hold for it to run, and what to do when the conditions are not met.

Predecessor-state conditions

Every task chained from a predecessor selects a state condition. The five values exposed in the workflow editor are:

State condition	Run task when predecessor…
`success or skipped`	…succeeded **or** was skipped by upstream branching (default)
`success`	…succeeded only
`error or cancelled`	…failed **or** was cancelled
`error`	…failed only — use for on-failure tasks
`any`	…finished with any outcome

> **Sources:** Build workflows (DT docs) — the page lists the five state-condition options verbatim and shows reading them as a sentence ("Run this task if <predecessor> ended with <state>").

"Else" behaviour

When a task's conditions are not met, the **else** behaviour decides what happens next:

- **Skip** — mark this task as skipped; downstream tasks evaluate their own conditions (a downstream task with `success or skipped` still runs; one with `success` will not).
- **Stop** — terminate the workflow run.

Custom conditions

For checks beyond predecessor state — inspecting a field on a prior task's result — use a **custom** condition expressed as a Jinja expression that must evaluate to a boolean:

```
{{ result("problem_details")["problem"]["riskAssessment"]["riskScore"] &gt;= 9 }}
```

The `result("task_name")` function returns the result object of a previously executed task; `event()` and `execution()` are also available. See Workflow reference / Jinja expressions (DT docs) for the full expression catalog.

Illustrative task YAML

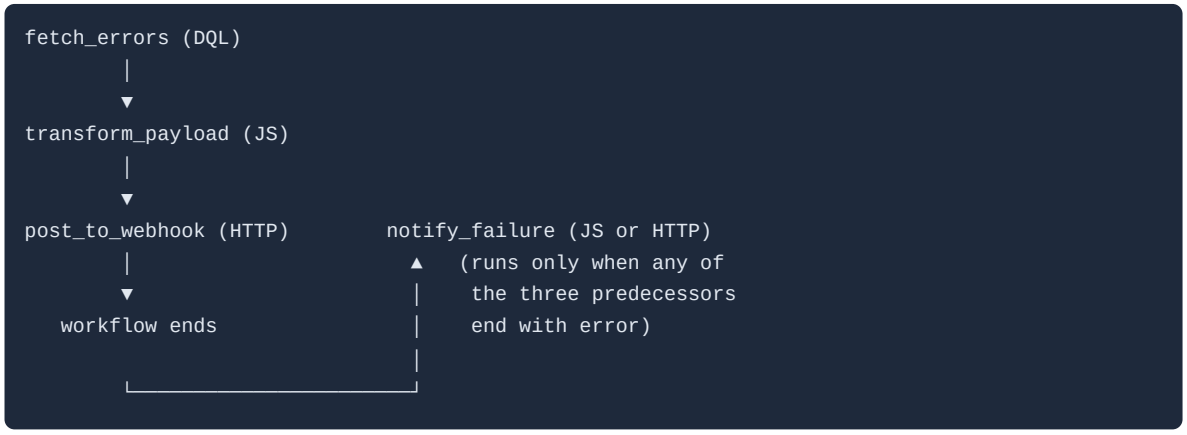
The shape below illustrates the conceptual structure of a failure-branching task. **Verify the exact YAML keys and casing in your tenant** — workflows are most commonly authored in the editor UI, and the on-disk YAML schema can drift between releases:

```
# Predecessor: a DQL query → JS transform chain
notify_failure:
  action: dynatrace.automations:run-javascript
  description: Notify on-call when any preceding task fails
  predecessors:
    - fetch_problem_data
    - transform_payload
    - post_to_webhook
  conditions:
    states:
      fetch_problem_data: error or cancelled
      transform_payload: error or cancelled
      post_to_webhook: error or cancelled
    custom: ""
  input:
    script: |
      export default async function({ execution, result }) {
        return { posted_failure_notice: true };
      }
```

> The literal keys `states` / `custom` and the string values shown reflect the field names surfaced by the docs assistant and community examples — they may differ in casing or nesting in the exported workflow JSON. If you are generating workflows programmatically, export a workflow you built in the UI and use that shape as the source of truth.

Worked example — DQL → JS → webhook with a failure-notify branch

A common shape: query data, transform it, send it somewhere, and send a separate notice if any step fails.



- **`fetch_errors`** runs a DQL query against logs/spans.
- **`transform_payload`** is a JavaScript task with predecessor `fetch_errors`, condition `states: { fetch_errors: success }`. If the query failed, this task does not run.
- **`post_to_webhook`** is an HTTP task with predecessor `transform_payload`, condition `states: { transform_payload: success }`.
- **`notify_failure`** has all three as predecessors with `states: <predecessor>: error or cancelled` and an **else: Skip**. It runs only when at least one upstream task ends with an error.

In community practice, the failure-notify branch is kept narrowly scoped — a single notification or a quick cleanup — rather than a parallel happy-path. If recovery logic itself can fail, give it its own `notify_failure` predecessor as well.

Decision guidance — JS try/catch vs workflow-level branching

Use try/catch inside a JS task when…	**Use workflow-level `conditions` when…**
The error is recoverable — retry, default value, skip a missing field	The error is terminal for this task — downstream work cannot proceed
The decision logic is localized to one API call or one parse step	The decision spans multiple tasks (DQL query, transform, post)
You want the task to **succeed with a degraded result**	You want the task to **fail and route around it**
The error category is predictable in code (network timeout, JSON parse)	The error could come from any of several task types (DQL, HTTP, JS, action)

The two patterns compose. A JS task can `try/catch` its own recoverable errors and still throw (re-throw or `throw new Error(...)`) when the failure is genuinely terminal — at which point the workflow's `conditions` graph takes over and routes to the on-failure branch.

Related notebooks

- **WFLOW-07: Auto-Remediation** — remediation tasks are the canonical use case for `error` / `error or cancelled` conditions. A diagnose task runs; a remediation task runs only on `error`; a verify task confirms the remediation; a notify task runs on `error or cancelled` of either remediation or verify.
- **WFLOW-05: Incident Management** — PagerDuty/ServiceNow tasks are commonly chained as on-failure branches off a primary workflow — incident creation runs only if the primary path errored out.

> **Sources:** Build workflows (DT docs), Workflow reference / Jinja expressions (DT docs). **Derived:** The decision table contrasting JS try/catch vs workflow conditions, and the recommendation to keep failure-notify branches narrowly scoped, synthesize the two cited pages with community-observed patterns from WFLOW-05 and WFLOW-07 — verify the on-failure shape in your own tenant before adopting at scale.

6. Working with Data

Transform DQL Results

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch logs, from: now() - 1h
        | filter loglevel == "ERROR"
        | fields timestamp, content, dt.entity.service
        | limit 100
      `
    }
  });

  const records = result.result.records || [];

  // Group by service
  const byService = records.reduce((acc, log) =&gt; {
    const service = log['dt.entity.service'] || 'unknown';
    if (!acc[service]) {
      acc[service] = [];
    }
    acc[service].push(log.content);
    return acc;
  }, {});

  // Create summary
  const summary = Object.entries(byService).map(([service, logs]) =&gt; ({
    service,
    error_count: logs.length,
    sample: logs[0]
  }));

  return {
    total_errors: records.length,
    by_service: summary
  };
}
```

Parse Event Tags

```
export default async function({ event }) {
  const tags = event().tags || [];

  // Parse key:value tags
  const tagMap = tags.reduce((acc, tag) =&gt; {
    if (tag.includes(':')) {
      const [key, value] = tag.split(':');
      acc[key] = value;
    } else {
      acc[tag] = true;
    }
    return acc;
  }, {});

  return {
    team: tagMap['team'] || 'unknown',
    env: tagMap['env'] || 'unknown',
    is_critical: tagMap['critical'] || false
  };
}
```

Build Dynamic Queries

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  const entityId = event().root_cause_entity_id;
  const severity = event().severity;

  // Build query based on entity type
  let query;
  if (entityId.startsWith('SERVICE-')) {
    query = `
      fetch spans, from: now() - 1h
      | filter dt.entity.service == "${entityId}"
      | filter span.status_code &gt;= 500
      | summarize errors = count()
    `;
  } else if (entityId.startsWith('HOST-')) {
    query = `
      fetch logs, from: now() - 1h
      | filter dt.entity.host == "${entityId}"
      | filter loglevel == "ERROR"
      | summarize errors = count()
    `;
  }

  const result = await queryExecutionClient.queryExecute({
    body: { query }
  });

  return {
    entity_type: entityId.split('-')[0],
    errors: result.result.records[0]?.errors || 0
  };
}
```

7. Passing Data Between Tasks

A workflow is a graph of tasks. Each task produces a result; downstream tasks read those results. Two access patterns — Jinja expressions for declarative task inputs, and the `result()` SDK function for JavaScript tasks — share the same underlying payload shape but differ in invocation.

7.1. The `result("task_name")` expression

Context	Form	Notes
Jinja (in any non-JS task's `input`)	`{{ result("task_name") }}`	Synchronous. Returns the task's full result object. Attribute access (`result("t").foo`) or `.get('foo', 'fallback')` for safer reads.

Context	Form	Notes
JavaScript task	`await result('task_name')`	**`result` is async in JS — forgetting `await` returns a Promise, not the data.** Imported from `@dynatrace-sdk/automation-utils`.

The returned payload shape is the same in both contexts; only the invocation differs.

> **Sources:** Workflow expression reference (DT docs), Run JavaScript Workflow Action (DT docs), Automation Utils SDK (Dynatrace Developer).

7.2. Result shape by task type

The accessor path depends on what the upstream task returned:

Upstream task type	Access path	Example
DQL query (`dynatrace.automations:execute-dql-query`)	`.records` — array of row objects	`{{ result("query_errors").records[0].error_count }}`
Run JavaScript (`dynatrace.automations:run-javascript`)	The returned object directly at top level	`{{ result("transform").affected_services }}`
HTTP request (`dynatrace.http:request`)	`.body` for response payload, `.statusCode` for status	`{{ result("post_webhook").statusCode }}`

> **The `.output` wrapper is not universal.** Some older notebook patterns reference `result("task").output` — that field only exists when the upstream task literally returns an object containing a key named `output` (e.g., a JS task whose function returns `{ output: ... }`). It is not a built-in wrapper. Verify against the specific task type's documented result shape rather than assuming `.output` is always present.

7.3. Worked example — DQL → JS transform → notification

A common three-task pattern: query a metric, transform the result into a notification-friendly shape, send it.

Task 1 — `fetch_errors` (DQL):

```
name: fetch_errors
action: dynatrace.automations:execute-dql-query
input:
  query: |
    fetch logs, from: now() - 15m
    | filter loglevel == "ERROR"
    | summarize error_count = count(), by: { dt.entity.service }
    | sort error_count desc
    | limit 5
```

Result shape: `{ records: [ { "dt.entity.service": "SERVICE-...", error_count: 42 }, ... ] }`

Task 2 — `transform` (JavaScript), reads `fetch_errors`:

```
import { result } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const upstream = await result('fetch_errors');
  const rows = upstream.records ?? [];

  if (rows.length === 0) {
    return { has_errors: false, summary: 'No errors in the last 15 minutes.' };
  }

  const top = rows[0];
  const summary_lines = rows.map(
    (r) =&gt; `- ${r['dt.entity.service']}: ${r.error_count} errors`
  );

  return {
    has_errors: true,
    top_service: top['dt.entity.service'],
    top_count: top.error_count,
    affected_count: rows.length,
    summary: `Top offender: ${top['dt.entity.service']} (${top.error_count} errors)`,
    summary_lines,
  };
}
```

Result shape: `{ has_errors: true, top_service: "SERVICE-...", top_count: 42, affected_count: 3, summary: "...", summary_lines: [...] }`

Task 3 — `notify_slack` (Slack message), reads `transform`:

```
name: notify_slack
action: dynatrace.slack:slack-send-message
conditions:
  custom: '{{ result("transform").has_errors }}'
input:
  channel: "#sre-alerts"
  message: |
    *Error spike detected*
    {{ result("transform").summary }}

    Affected services ({{ result("transform").affected_count }}):
    {% for line in result("transform").summary_lines %}{{ line }}
    {% endfor %}
```

The `conditions.custom` Jinja expression gates the notification: if `transform` returned `has_errors: false`, this task is skipped.

7.4. Object passing — arrays, nested objects, flatten vs nested

JSON-serializable values pass cleanly between tasks: primitives, arrays, plain objects, nested combinations. There is no special encoding step.

When to keep nested:
- Downstream JS tasks consume the data (object access is ergonomic).
- The shape is logically grouped — `{ problem: { id, severity }, entities: [...] }` is clearer than ten flat fields.

When to flatten:
- Downstream Jinja templates read the data — deep paths like `{{ result("t").problem.entities[0].tags[2] }}` are fragile and unreadable.
- The notification channel has flat-key templating (e.g., a webhook expecting `entity_id` not `entities[0].id`).

Rule of thumb: flatten at the **producer** side (in the JS transform), not at the consumer side. A transform task that emits `{ top_service, top_count, summary }` is easier for a downstream notification template to consume than one that emits `{ raw_records: [...] }`.

Size budget: task results are passed through the workflow execution store and shown in run history. Returning multi-megabyte payloads inflates run history and can hit task-result size limits. If a DQL query returns 10,000 rows, summarize in the JS task (top 5, counts, p95) — do not pass the full record set downstream.

7.5. Variable scoping — what's available where

Source	Jinja	JavaScript	Scope
Trigger event	`{{ event() }}`, `{{ event()['display_id'] }}`	`const ex = await execution(); ex.params.event`	Workflow-wide — every task reads the same event

Source	Jinja	JavaScript	Scope
Previous task result	`{{ result("task_name") }}`	`await result('task_name')`	Workflow-wide once the task has completed
Workflow inputs (parameters)	`{{ execution().params.<name> }}`	`(await execution()).params.<name>`	Workflow-wide; set at trigger time
Secrets / env	`{{ env.MY_SECRET }}`	`env.MY_SECRET` (passed via task input destructure)	Workflow-wide; resolved at task execution
Local variables inside one task	(not exported)	`const x = ...` inside `export default async function() { ... }`	Task-local — **not visible to other tasks unless returned**

A task's `return` value is the **only** way to make data visible downstream. There is no shared mutable state between tasks — every cross-task value travels through `result()`.

7.6. Common pitfalls

Pitfall	Symptom	Fix
Forgetting `await` on `result()` in JS	Code gets a `Promise<Result>`, not the data; `.records` is `undefined`	Always `await result('task')`
Assuming `.output` is universal	`result("dql_task").output` returns `undefined`	Use `.records` for DQL, top-level fields for JS, `.body` / `.statusCode` for HTTP
Hyphenated/dotted event keys in attribute syntax	`event().security-problem.technology` is parsed as subtraction	Use bracket form: `event()['security-problem.technology']`
`default` filter with missing parent	`result("t").foo | default("x")` still errors if `result("t")` itself is undefined	Use `.get()`: `result("t").get("foo", "x")`
Reading a task that may have errored	Downstream task fails with `Undefined variables`	Add a `conditions.states` gate (`success` only) or check `.get()` with a fallback
Reading a skipped task's result	A task gated by an unmet condition is *skipped*, not *failed*; `result("skipped_task")` may return an empty / undefined result	Branch the downstream task's `conditions` on the same predicate, not on the skipped task's result

Pitfall	Symptom	Fix
Returning non-JSON-serializable values from JS (e.g., a `Date`, a class instance)	Silent coercion to string or `{}`	Return plain objects: `new Date().toISOString()`, plain literals
Returning huge payloads	Slow run history, possible truncation	Summarize in the producer task; return only what downstream tasks need

7.7. When this pattern doesn't fit

For long-running multi-step orchestrations where intermediate state must persist *outside* a single workflow run (resumable workflows, cross-workflow handoffs), workflow `result()` is the wrong tool — it lives only for the duration of one execution. Persistent state belongs in **Document Service** (for structured documents) or **Settings 2.0** (for configuration-shaped state).

> **Sources:** Workflow expression reference (DT docs), Run JavaScript Workflow Action (DT docs), Automation Utils SDK (Dynatrace Developer). **Derived:** the §6.2 result-shape table consolidates per-task-type access paths from the SDK reference and community examples; the §6.4 flatten-at-producer rule and the §6.6 pitfall list combine the cited expression-reference pitfalls with patterns observed across WFLOW-05/07/08 — verify exact shapes against your tenant's task documentation before deploying at scale.

8. Integration Examples

GitHub Issue Creation

```
export default async function({ event, env }) {
  const response = await fetch(
    `https://api.github.com/repos/${env.GITHUB_REPO}/issues`,
    {
      method: 'POST',
      headers: {
        'Authorization': `token ${env.GITHUB_TOKEN}`,
        'Accept': 'application/vnd.github.v3+json',
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        title: `[Dynatrace] ${event().title}`,
        body: `
&lt;a id="problem-details"&gt;&lt;/a&gt;
## Problem Details
- **Problem ID:** ${event().display_id}
- **Severity:** ${event().severity}
- **Started:** ${event().start_time}
- **Link:** [View in Dynatrace](${event().problem_url})

&lt;a id="affected-entities"&gt;&lt;/a&gt;
## Affected Entities
${event().affected_entity_ids.map(e =&gt; `- ${e}`).join('\n')}
        `,
        labels: ['dynatrace', event().severity.toLowerCase()]
      })
    }
  );

  const issue = await response.json();
  return { issue_number: issue.number, url: issue.html_url };
}
```

Datadog Event

```
export default async function({ event, env }) {
  const response = await fetch('https://api.datadoghq.com/api/v1/events', {
    method: 'POST',
    headers: {
      'DD-API-KEY': env.DATADOG_API_KEY,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      title: event().title,
      text: `Problem ${event().display_id} detected by Dynatrace`,
      alert_type: event().severity === 'CRITICAL' ? 'error' : 'warning',
      source_type_name: 'dynatrace',
      tags: ['source:dynatrace', `severity:${event().severity}`]
    })
  });

  return await response.json();
}
```

Splunk HEC Event

```
export default async function({ event, env }) {
  const response = await fetch(
    `${env.SPLUNK_HEC_URL}/services/collector/event`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Splunk ${env.SPLUNK_HEC_TOKEN}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        event: {
          problem_id: event().display_id,
          title: event().title,
          severity: event().severity,
          affected_entities: event().affected_entity_ids
        },
        sourcetype: 'dynatrace:problem',
        index: 'observability'
      })
    }
  );

  return { sent: response.ok };
}
```

9. Performance Tips

Parallel Execution

```
export default async function({ event }) {
  const entityIds = event().affected_entity_ids;

  // Execute all lookups in parallel
  const results = await Promise.all(
    entityIds.map(id =&gt; getEntityDetails(id))
  );

  return { entities: results };
}
```

Limit Query Scope

```
// Bad: Fetching too much data
const result = await queryExecutionClient.queryExecute({
  body: {
    query: `fetch logs, from: now() - 7d | filter loglevel == "ERROR"`
  }
});

// Good: Limited time range and fields
const result = await queryExecutionClient.queryExecute({
  body: {
    query: `
      fetch logs, from: now() - 1h
      | filter loglevel == "ERROR"
      | fields timestamp, content
      | limit 100
    `,
    requestTimeoutMilliseconds: 30000
  }
});
```

Use Timeouts

```
// Add timeout to external calls
const controller = new AbortController();
const timeout = setTimeout(() =&gt; controller.abort(), 10000);

try {
  const response = await fetch(url, {
    signal: controller.signal,
    // ... other options
  });
  return await response.json();
} finally {
  clearTimeout(timeout);
}
```

10. Configuring Task Timeouts

Two timeouts apply to every workflow task — see WFLOW-01 § *Platform Limits* for the conceptual split between **task timeout** (wall-clock budget for the whole task) and **Dynatrace runtime timeout** (per-action budget, 120s default, applies to DQL queries and individual JS/HTTP actions). This section covers the configuration shape on each of the three task types most authors touch.

`timeout` field — units and shape

The `timeout` field on a workflow task is the **task timeout**. It accepts an integer count of **seconds** when configured in JSON/YAML via the AutomationEngine. The Workflows UI exposes it under **Options → Adapt timeout** and labels the field *Timeout this task (seconds)*. Default: 60 minutes (3600s). Maximum: 7 days (604800s).

Some task types (e.g., `wait-for-event` used in approval flows) accept a **duration string** like `"30m"` or `"24h"` on the task's own `timeout` / `waitFor` field. That is *not* the same field as the AutomationEngine task timeout — it's specific to the wait action. See the approval example below.

On `execute-dql-query`

```
tasks:
  - name: query_errors
    description: Count error logs in the last hour
    action: dynatrace.automations:execute-dql-query
    timeout: 180   # task timeout, seconds — caps the whole task
    input:
      query: |
        fetch logs, from: now() - 1h
        | filter loglevel == "ERROR"
        | summarize errors = count()
      # Note: the DQL engine's own 120s runtime budget still applies
      # to the query execution itself. Raising `timeout` above 120s
      # only helps if the task does more than execute one query.
```

On `run-javascript`

```
tasks:
  - name: enrich_problem
    action: dynatrace.automations:run-javascript
    timeout: 300   # task timeout, seconds
    input:
      script: |
        import { queryExecutionClient } from '@dynatrace-sdk/client-query';
        export default async function() {
          const result = await queryExecutionClient.queryExecute({
            body: {
              query: `fetch logs, from: now() - 1h | filter loglevel == "ERROR" | summarize
count()`,
              // This is the per-request HTTP timeout from JS to DQL engine,
              // in milliseconds. Distinct from both the task timeout above
              // and the 120s runtime budget.
              requestTimeoutMilliseconds: 30000
            }
          });
          return { count: result.result.records[0]?.['count()'] || 0 };
        }
```

Inside a JS task you actually deal with **three** timeouts: the task `timeout`, the runtime's 120s per-action ceiling, and any `requestTimeoutMilliseconds` you pass to SDK calls. The first one budgets the task as a whole; the second one is enforced by the runtime regardless; the third one is yours to set per outbound call.

On approval / wait-for-event tasks

Approval tasks intentionally wait for a human signal — task timeouts here are long by design, and the `wait-for-event` action has its own `timeout` field that accepts a duration string:

```
tasks:
  - name: wait_for_approval
    action: dynatrace.automations:wait-for-event
    input:
      eventType: "remediation.approval"
      timeout: "30m"     # wait-action's own timeout — duration string
      correlationId: "{{ event()['display_id'] }}"
    # The outer AutomationEngine task timeout should be set &gt;= the wait timeout:
    timeout: 2400        # 40 minutes in seconds, gives the wait 30m of headroom
```

See WFLOW-07 § *Approval Workflows* for the full human-in-the-loop pattern.

Decision guidance: raise the timeout, or fix the query?

When a DQL or JavaScript task hits a timeout, the right move depends on **which** timeout fired:

Symptom	What hit	First move
Task fails at ~120s with a runtime/engine error	Dynatrace runtime timeout (per-action)	Narrow the query window (`from: now() - 15m` instead of `now() - 24h`), pre-aggregate into a metric or bizevent, or split into multiple smaller tasks. Raising the task `timeout` will not help.
Task fails at exactly the configured task `timeout` value	Task timeout	Raise `timeout` , or break the task into smaller tasks. Verify the work genuinely needs the budget — most legitimately-long tasks are waiting on a human or external system, not computing.
`queryExecute()` returns a request-timeout error well before 120s	`requestTimeoutMilliseconds` too low	Raise it on the SDK call. Default to 30000 (30s); raise to 60000 only if the query genuinely needs longer (and consider whether the query should be pre-aggregated instead).
Approval/wait task fires at its `timeout` value	Wait-action's own timeout	This is usually the correct behavior — the human did not respond. Decide whether to escalate, auto-approve, or fail.

> **Rule of thumb.** Raise timeouts only after narrowing scope. A task that needs 10 minutes of DQL is almost always a task that needs a pre-aggregation upstream — fix that first.
>
> **Sources:** Build workflows — Adapt timeout (DT docs), Run JavaScript action (DT docs), Dynatrace SDK — client-query (Dynatrace Developer). **Derived:** the three-timeout interaction model in *On `run-javascript`* and the decision table combine the cited per-field documentation; no single source presents them together.

Monitor JavaScript Task Performance

```
// JavaScript task execution times
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| filter dt.automation_engine.action.function == "run-javascript"
| summarize {executions = count(), avg_ms = avg(duration) / 1ms, p95_ms = percentile(duration, 95) / 1ms}, by:{dt.automation_engine.workflow.title}
| sort p95_ms desc
| limit 20
```

```
// An empty result here is the HEALTHY answer: on the validation tenant http-function ran 49
// times over 7 days with 0 ERROR states. Rows appearing means real HTTP task failures.
// HTTP request task errors
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling —
those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED ·
SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                    true only on terminal records — filter on it,
otherwise a
//                                        single execution is counted once as RUNNING and
again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.action.function == "http-function"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title, dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

11. CMDB-Driven Host Tag Enrichment

A frequent ask: read host→application mappings from a CMDB and apply them as host tags in bulk. This is the capstone that combines this notebook's building blocks — an SDK DQL query (§2), `fetch()` to a Dynatrace API with auth (§3), retry logic (§4), `result()` / `withItems` data passing between tasks (§7), and the Credential Vault — into one deployable workflow.

The shape is two `run-javascript` tasks:

1. **`query_hosts_and_enrich`** — enriches `dt.entity.host` through a CMDB lookup chain (`lookup [load "/lookups/…"]`), dedups by host id, and diffs against existing tags → returns `hosts[]` .
2. **`apply_tags_to_hosts`** — loops (`withItems`) over `hosts[]` and POSTs a `set hostTag` operation to the **OneAgent Remote Configuration Management API** (`${ENVIRONMENT_URL}/api/v2/oneagents/remoteConfigurationManagement`) using a Gen2 (classic) `oneAgents.write` API token read from the Credential Vault. Guardrails: `DRY_RUN` , `MAX_HOSTS` , skip-

existing, 409-retry. (Config is supplied via workflow inputs; the platform-token path, `fleet-management:oneagents:write`, isn't GA yet.)

One `set hostTag` operation writes both `primary_tags.*` (the prefix is mandatory) and `dt.*` primary fields (`dt.security_context`, `dt.cost.costcenter`) — see **FAQ-02 (Tagging: Sources, Standards & Strategy)** for the tagging-source model.

> **Hands-on build:** the complete step-by-step walkthrough — build it in the Workflows editor, both scripts, customization, the loop/condition wiring, the safety model, and an import-ready YAML skeleton — is in the **WFLOW-95 LAB: CMDB-Driven Host Tag Enrichment**.
>
> **Sources:** OneAgent remote configuration management API — POST a configuration job (DT docs), Lookup data in Grail (DT docs). **Derived:** the two-task shape combines this notebook's §2–§7 mechanics — full hands-on build in the WFLOW-95 LAB.

Next Steps

With custom integrations ready, learn governance:

Recommended Path

1. **WFLOW-09: Security, Governance & Monitoring** - Production best practices

Key Takeaways

- **JavaScript actions** enable custom logic
- **Dynatrace SDK** provides typed API access
- **HTTP requests** integrate any REST API
- **Error handling** is critical for reliability — inside a task use `try/catch`; between tasks use the workflow `conditions` block
- **On-failure branches** use `error` / `error or cancelled` state conditions to route around dead tasks
- **Performance** matters - limit query scope, use parallelism
- **Bulk host operations** combine an enrichment query with a `withItems` loop over the Remote Configuration Management API — introduced in §11, full hands-on build in the **WFLOW-95 LAB**

Summary

In this notebook, you learned:

- JavaScript action structure and context
- Dynatrace SDK clients and methods

- HTTP request patterns (GET, POST, auth)
- Error handling and retry logic
- Workflow-level failure branching with task `conditions` (predecessor state and custom Jinja)
- Data transformation patterns
- Integration examples (GitHub, Datadog, Splunk)
- Performance optimization tips
- A CMDB-driven host tag enrichment capstone (§11) — full build in the **WFLOW-95 LAB**

References

- Run JavaScript action (DT docs)
- HTTP request action (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Dynatrace Query Language reference (DT docs)
- OneAgent remote configuration management API — POST a configuration job (DT docs)
- Lookup data in Grail (DT docs)
- Dynatrace SDK (Dynatrace Developer)
- Dynatrace Developer Portal (Dynatrace)
